Project 07: Banking System

Difficulty: Advanced | Estimated Time: 3 Weeks

---

1. Project Overview and Goals

This project models a core banking system — one of the most demanding database domains in software engineering. It covers customer onboarding, multi-type accounts (savings, checking, credit, loan), transactional ledger (double-entry bookkeeping), wire transfers, interest calculations, fraud detection signals, and regulatory reporting. Banking databases must guarantee ACID properties above all else.

Goals:

- Implement double-entry bookkeeping with a debit/credit ledger.
- Enforce atomicity in fund transfers using PostgreSQL transactions.
- Model compound interest calculations for savings and loan accounts.
- Build a fraud detection signal system using velocity checks.
- Implement row-level security (RLS) for multi-tenant data isolation.
- Practice advisory locks and optimistic concurrency for account updates.

---

2. Learning Objectives

- Understand why banking uses double-entry bookkeeping and implement it in SQL.
- Use `SERIALIZABLE` isolation level for critical financial transactions.
- Implement `SELECT FOR UPDATE NOWAIT` to prevent deadlocks.
- Use `pg_advisory_lock` for application-level locking.
- Write compound interest calculation functions.
- Implement Row-Level Security (RLS) policies.
- Build a fraud detection velocity check system.
- Practice `NUMERIC` precision arithmetic — never use FLOAT for money.
- Understand deferred constraints and transaction-level enforcement.

---

3. Functional Requirements

- **Customers**: KYC information, identity verification, risk tier.
- **Accounts**: Savings, checking, credit, loan — each with own rules.
- **Ledger**: Every financial event creates debit + credit entries.
- **Transfers**: Internal (between own accounts), external (wire), interbank.
- **Interest**: Daily accrual, monthly posting for savings; amortization for loans.
- **Statements**: Monthly account statements with running balance.
- **Fraud**: Velocity checks (too many transactions, large amounts).
- **RLS**: Customers can only see their own accounts.

---

4. Non-Functional Requirements

- All money in `NUMERIC(18,4)` — 4 decimal places for interest precision.
- Every transfer is atomic — either both legs post or neither does.
- Ledger entries are IMMUTABLE — no updates or deletes permitted.
- Account balance is NEVER stored — always derived from ledger sum.

- Regulatory: All transactions retained for 7 years.
- ISO 4217 currency codes enforced via CHECK constraint.

---

5. Complete Database Schema

```
-- ============================================================
-- BANKING SYSTEM - Complete Schema
-- PostgreSQL 15+
-- ============================================================

CREATE SCHEMA IF NOT EXISTS bank;
SET search_path = bank, public;

-- Money type: always NUMERIC(18,4), never FLOAT
CREATE DOMAIN money_amount AS NUMERIC(18,4) CHECK (VALUE IS NOT NULL);
CREATE DOMAIN currency_code AS CHAR(3) CHECK (VALUE ~ '^[A-Z]{3}$');

-- ------------------------------------------------------------
-- BRANCHES
-- ------------------------------------------------------------
CREATE TABLE branches (
    branch_id    SERIAL PRIMARY KEY,
    code         VARCHAR(10) NOT NULL UNIQUE,
    name         VARCHAR(200) NOT NULL,
    address      TEXT,
    city         VARCHAR(100),
    country      VARCHAR(100) NOT NULL DEFAULT 'USA',
    swift_code   VARCHAR(11),
    routing_num  VARCHAR(20),
    is_active    BOOLEAN NOT NULL DEFAULT TRUE
);

INSERT INTO bank.branches (code, name, city) VALUES
  ('HQ-001',  'Main Branch',         'New York'),
  ('BR-002',  'Chicago Branch',      'Chicago'),
  ('BR-003',  'San Francisco Branch','San Francisco');

-- ------------------------------------------------------------
-- CUSTOMERS
-- ------------------------------------------------------------
CREATE TABLE customers (
    customer_id    SERIAL PRIMARY KEY,
    customer_number VARCHAR(20) NOT NULL UNIQUE,  -- e.g. CUST-0000001
    first_name     VARCHAR(100) NOT NULL,
    last_name      VARCHAR(100) NOT NULL,
    date_of_birth  DATE NOT NULL,
    ssn_hash       VARCHAR(256),      -- hashed, never plaintext
    email          VARCHAR(300) NOT NULL UNIQUE,
    phone          VARCHAR(30),
    address        TEXT,
```

```
    city            VARCHAR(100),
    country         VARCHAR(100) NOT NULL DEFAULT 'USA',
    kyc_status      VARCHAR(20) NOT NULL DEFAULT 'Pending'
                    CHECK (kyc_status IN ('Pending','Verified','Failed','Expired')),
    risk_tier       VARCHAR(10) NOT NULL DEFAULT 'Standard'
                    CHECK (risk_tier IN ('Low','Standard','High','Blocked')),
    branch_id       INTEGER NOT NULL REFERENCES branches(branch_id),
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_customers_number ON customers (customer_number);
CREATE INDEX idx_customers_email  ON customers (email);
CREATE INDEX idx_customers_kyc    ON customers (kyc_status);

-- ------------------------------------------------------------
-- ACCOUNT TYPES
-- ------------------------------------------------------------
CREATE TABLE account_types (
    type_id         SERIAL PRIMARY KEY,
    code            VARCHAR(20) NOT NULL UNIQUE,
    name            VARCHAR(100) NOT NULL,
    interest_rate   NUMERIC(6,4) NOT NULL DEFAULT 0.0000,  -- annual
    overdraft_limit money_amount NOT NULL DEFAULT 0.0000,
    min_balance     money_amount NOT NULL DEFAULT 0.0000,
    transaction_limit_daily money_amount,
    description     TEXT
);

INSERT INTO bank.account_types (code, name, interest_rate, overdraft_limit,
min_balance, transaction_limit_daily) VALUES
  ('SAVINGS',   'Savings Account',     0.0450, 0.0000,    500.0000, 10000.0000),
  ('CHECKING',  'Checking Account',    0.0100, 500.0000,    0.0000, 50000.0000),
  ('PREMIUM',   'Premium Savings',     0.0650, 0.0000,   5000.0000, 50000.0000),
  ('CREDIT',    'Credit Card Account', 0.1999, 5000.0000,  0.0000, 25000.0000),
  ('LOAN',      'Personal Loan',       0.0899, 0.0000,     0.0000, NULL);

-- ------------------------------------------------------------
-- ACCOUNTS
-- ------------------------------------------------------------
CREATE TABLE accounts (
    account_id      SERIAL PRIMARY KEY,
    account_number  VARCHAR(20) NOT NULL UNIQUE,  -- e.g. ACC-000000001
    customer_id     INTEGER NOT NULL REFERENCES customers(customer_id),
    type_id         INTEGER NOT NULL REFERENCES account_types(type_id),
    branch_id       INTEGER NOT NULL REFERENCES branches(branch_id),
    currency        currency_code NOT NULL DEFAULT 'USD',
    status          VARCHAR(20) NOT NULL DEFAULT 'Active'
                    CHECK (status IN ('Active','Frozen','Closed','Suspended')),
    opened_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    closed_at       TIMESTAMPTZ,
    last_activity   TIMESTAMPTZ,
```

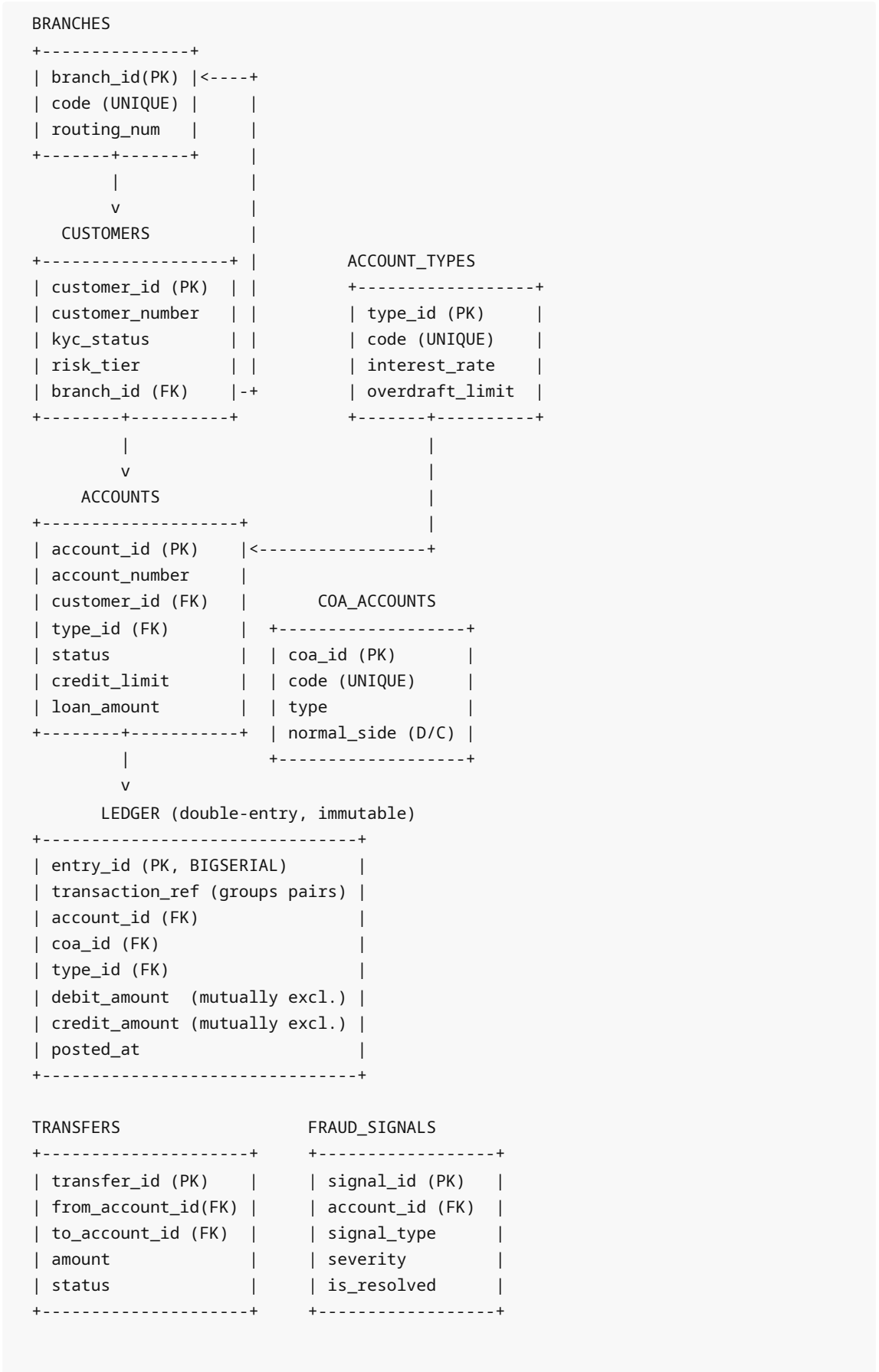
```
    credit_limit    money_amount,   -- for credit accounts
    loan_amount     money_amount,   -- for loan accounts
    loan_term_months SMALLINT,
    interest_rate   NUMERIC(6,4),   -- can override account type rate
    CONSTRAINT chk_closed_has_date CHECK (
        (status = 'Closed' AND closed_at IS NOT NULL) OR status <> 'Closed'
    )
);

COMMENT ON TABLE accounts IS 'Account balance is never stored — always computed from
ledger';

CREATE INDEX idx_accounts_customer ON accounts (customer_id);
CREATE INDEX idx_accounts_number   ON accounts (account_number);
CREATE INDEX idx_accounts_status   ON accounts (status) WHERE status = 'Active';

-- ------------------------------------------------------------
-- CHART OF ACCOUNTS (double-entry)
-- ------------------------------------------------------------
CREATE TABLE coa_accounts (
    coa_id       SERIAL PRIMARY KEY,
    code         VARCHAR(20) NOT NULL UNIQUE,
    name         VARCHAR(200) NOT NULL,
    type         VARCHAR(20) NOT NULL
                 CHECK (type IN ('Asset','Liability','Equity','Revenue','Expense')),
    normal_side  CHAR(1) NOT NULL CHECK (normal_side IN ('D','C')),
    -- D=Debit normal, C=Credit normal
    description  TEXT
);

INSERT INTO bank.coa_accounts (code, name, type, normal_side) VALUES
  ('1000', 'Cash and Due from Banks',       'Asset',     'D'),
  ('1100', 'Customer Deposits - Checking',  'Liability', 'C'),
  ('1200', 'Customer Deposits - Savings',   'Liability', 'C'),
  ('1300', 'Customer Deposits - Premium',   'Liability', 'C'),
  ('2000', 'Loans Receivable',              'Asset',     'D'),
  ('2100', 'Credit Card Receivable',        'Asset',     'D'),
  ('3000', 'Interest Income',               'Revenue',   'C'),
  ('4000', 'Interest Expense',              'Expense',   'D'),
  ('5000', 'Fee Income',                    'Revenue',   'C'),
  ('6000', 'Wire Transfer Clearing',        'Asset',     'D');

-- ------------------------------------------------------------
-- TRANSACTION TYPES
-- ------------------------------------------------------------
CREATE TABLE transaction_types (
    type_id      SERIAL PRIMARY KEY,
    code         VARCHAR(30) NOT NULL UNIQUE,
    name         VARCHAR(100) NOT NULL,
    description  TEXT
);
```

```sql
INSERT INTO bank.transaction_types (code, name) VALUES
  ('DEPOSIT',      'Cash Deposit'),
  ('WITHDRAWAL',   'Cash Withdrawal'),
  ('TRANSFER_IN',  'Incoming Transfer'),
  ('TRANSFER_OUT', 'Outgoing Transfer'),
  ('INTEREST',     'Interest Credit'),
  ('FEE',          'Service Fee'),
  ('LOAN_DISBUR',  'Loan Disbursement'),
  ('LOAN_REPAY',   'Loan Repayment'),
  ('WIRE_IN',      'Incoming Wire'),
  ('WIRE_OUT',     'Outgoing Wire'),
  ('REVERSAL',     'Transaction Reversal');

-- ------------------------------------------------------------
-- LEDGER (immutable double-entry)
-- ------------------------------------------------------------
CREATE TABLE ledger (
    entry_id      BIGSERIAL PRIMARY KEY,
    transaction_ref VARCHAR(40) NOT NULL,  -- groups debit + credit pair
    account_id    INTEGER NOT NULL REFERENCES accounts(account_id),
    coa_id        INTEGER REFERENCES coa_accounts(coa_id),
    type_id       INTEGER NOT NULL REFERENCES transaction_types(type_id),
    debit_amount  money_amount NOT NULL DEFAULT 0.0000,
    credit_amount money_amount NOT NULL DEFAULT 0.0000,
    balance_after money_amount,   -- snapshot for statement generation
    currency      currency_code NOT NULL DEFAULT 'USD',
    description   VARCHAR(500),
    reference_id  BIGINT,        -- transfer, wire, loan ID
    reference_type VARCHAR(30),
    posted_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    value_date    DATE NOT NULL DEFAULT CURRENT_DATE,
    created_by    VARCHAR(100) NOT NULL DEFAULT current_user,
    CONSTRAINT chk_one_side_only CHECK (
        (debit_amount > 0 AND credit_amount = 0)
        OR (credit_amount > 0 AND debit_amount = 0)
    )
);

COMMENT ON TABLE ledger IS 'Immutable financial ledger — no updates or deletes
allowed';

CREATE INDEX idx_ledger_account   ON ledger (account_id, posted_at DESC);
CREATE INDEX idx_ledger_ref       ON ledger (transaction_ref);
CREATE INDEX idx_ledger_date      ON ledger (value_date);
CREATE INDEX idx_ledger_type      ON ledger (type_id);

-- Partition ledger by year for performance
CREATE TABLE ledger_2024 PARTITION OF ledger FOR VALUES FROM ('2024-01-01') TO
('2025-01-01');
CREATE TABLE ledger_2025 PARTITION OF ledger FOR VALUES FROM ('2025-01-01') TO
('2026-01-01');
```

```sql
-- Re-declare as partitioned
-- Note: in production, create the parent as PARTITION BY RANGE at table creation
time

-- ------------------------------------------------------------
-- TRANSFERS
-- ------------------------------------------------------------
CREATE TABLE transfers (
    transfer_id    BIGSERIAL PRIMARY KEY,
    ref_number     VARCHAR(40) NOT NULL UNIQUE DEFAULT gen_random_uuid()::TEXT,
    from_account_id INTEGER NOT NULL REFERENCES accounts(account_id),
    to_account_id   INTEGER NOT NULL REFERENCES accounts(account_id),
    amount         money_amount NOT NULL CHECK (amount > 0),
    currency       currency_code NOT NULL DEFAULT 'USD',
    fee_amount     money_amount NOT NULL DEFAULT 0.0000,
    status         VARCHAR(20) NOT NULL DEFAULT 'Pending'
                   CHECK (status IN ('Pending','Completed','Failed','Reversed')),
    description    VARCHAR(500),
    initiated_by   VARCHAR(100),
    initiated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at   TIMESTAMPTZ,
    CONSTRAINT chk_no_self_transfer CHECK (from_account_id <> to_account_id)
);

CREATE INDEX idx_transfers_from   ON transfers (from_account_id);
CREATE INDEX idx_transfers_to     ON transfers (to_account_id);
CREATE INDEX idx_transfers_status ON transfers (status) WHERE status = 'Pending';

-- ------------------------------------------------------------
-- FRAUD SIGNALS
-- ------------------------------------------------------------
CREATE TABLE fraud_signals (
    signal_id    BIGSERIAL PRIMARY KEY,
    account_id   INTEGER NOT NULL REFERENCES accounts(account_id),
    signal_type  VARCHAR(50) NOT NULL,
    severity     VARCHAR(10) NOT NULL DEFAULT 'Medium'
                 CHECK (severity IN ('Low','Medium','High','Critical')),
    description  TEXT,
    raw_data     JSONB,
    is_resolved  BOOLEAN NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_fraud_account    ON fraud_signals (account_id, created_at DESC);
CREATE INDEX idx_fraud_unresolved ON fraud_signals (severity) WHERE is_resolved =
FALSE;
```

---

6. ASCII ER Diagram


BRANCHES
branch_id(PK)
code (UNIQUE)
routing_num
CUSTOMERS
customer_id (PK)
customer_number
kyc_status
risk_tier
branch_id (FK)
ACCOUNT_TYPES
type_id (PK)
code (UNIQUE)
interest_rate
overdraft_limit
ACCOUNTS
account_id (PK)
account_number
customer_id (FK)
type_id (FK)
status
credit_limit
loan_amount
COA_ACCOUNTS
coa_id (PK)
code (UNIQUE)
type
normal_side (D/C)
LEDGER (double-entry, immutable)
entry_id (PK, BIGSERIAL)
transaction_ref (groups pairs)
account_id (FK)
coa_id (FK)
type_id (FK)
debit_amount (mutually excl.)
credit_amount (mutually excl.)
posted_at
TRANSFERS
transfer_id (PK)
from_account_id(FK)
to_account_id (FK)
amount
status
FRAUD_SIGNALS
signal_id (PK)
account_id (FK)
signal_type
severity
is_resolved

```
TRANSACTION_TYPES
+------------------+
| type_id (PK)     |
| code (UNIQUE)    |
| name             |
+------------------+
```

---

7. Sample Data Scripts

```sql
SET search_path = bank, public;

-- Customers
INSERT INTO customers (customer_number, first_name, last_name, date_of_birth, email,
phone, kyc_status, risk_tier, branch_id) VALUES
  ('CUST-0000001', 'Alice',   'Johnson', '1985-03-15', 'alice.j@bank.com',  '212-
555-0101', 'Verified', 'Standard', 1),
  ('CUST-0000002', 'Robert',  'Williams','1972-07-22', 'robert.w@bank.com', '312-
555-0202', 'Verified', 'Low',      2),
  ('CUST-0000003', 'Maria',   'Garcia',  '1990-11-08', 'maria.g@bank.com',  '415-
555-0303', 'Verified', 'High',     3),
  ('CUST-0000004', 'James',   'Brown',   '1965-05-30', 'james.b@bank.com',  '212-
555-0404', 'Verified', 'Standard', 1);

-- Accounts
INSERT INTO accounts (account_number, customer_id, type_id, branch_id, currency,
interest_rate) VALUES
  ('ACC-000000001', 1, 1, 1, 'USD', 0.0450),  -- Alice Savings
  ('ACC-000000002', 1, 2, 1, 'USD', 0.0100),  -- Alice Checking
  ('ACC-000000003', 2, 3, 2, 'USD', 0.0650),  -- Robert Premium Savings
  ('ACC-000000004', 3, 2, 3, 'USD', 0.0100),  -- Maria Checking
  ('ACC-000000005', 4, 5, 1, 'USD', 0.0899);  -- James Loan

-- Ledger entries (opening balances)
-- Alice opens savings with $10,000 deposit
INSERT INTO ledger (transaction_ref, account_id, type_id, credit_amount,
description, value_date)
VALUES ('OPEN-ACC-001-' || NOW()::TEXT, 1, 1, 10000.0000, 'Opening deposit',
CURRENT_DATE);

INSERT INTO ledger (transaction_ref, account_id, type_id, credit_amount,
description, value_date)
VALUES ('OPEN-ACC-002-' || NOW()::TEXT, 2, 1, 2500.0000, 'Opening deposit',
CURRENT_DATE);

INSERT INTO ledger (transaction_ref, account_id, type_id, credit_amount,
description, value_date)
VALUES ('OPEN-ACC-003-' || NOW()::TEXT, 3, 1, 25000.0000, 'Opening deposit',
CURRENT_DATE);

INSERT INTO ledger (transaction_ref, account_id, type_id, credit_amount,
```

```
description, value_date)
VALUES ('OPEN-ACC-004-' || NOW()::TEXT, 4, 1, 1500.0000, 'Opening deposit',
CURRENT_DATE);

-- Sample transactions
INSERT INTO ledger (transaction_ref, account_id, type_id, debit_amount, description,
value_date)
VALUES ('TXN-W-001', 2, 2, 500.0000, 'ATM Withdrawal', CURRENT_DATE - 5);

INSERT INTO ledger (transaction_ref, account_id, type_id, credit_amount,
description, value_date)
VALUES ('TXN-D-001', 1, 1, 2000.0000, 'Direct deposit - payroll', CURRENT_DATE - 3);
```

---

8. Core SQL Queries

```
SET search_path = bank, public;

-- ------------------------------------------------------
-- Q1: Current account balance (derived from ledger)
-- ------------------------------------------------------
SELECT
    a.account_number,
    c.first_name || ' ' || c.last_name     AS customer,
    at2.name                               AS account_type,
    a.currency,
    SUM(l.credit_amount - l.debit_amount)  AS balance,
    COUNT(l.entry_id)                      AS total_entries,
    MAX(l.posted_at)::DATE                 AS last_transaction
FROM accounts a
JOIN customers c        ON a.customer_id  = c.customer_id
JOIN account_types at2  ON a.type_id      = at2.type_id
LEFT JOIN ledger l      ON a.account_id   = l.account_id
WHERE a.status = 'Active'
GROUP BY a.account_id, a.account_number, c.first_name, c.last_name, at2.name,
a.currency
ORDER BY a.account_number;

-- ------------------------------------------------------
-- Q2: Account statement with running balance
-- ------------------------------------------------------
SELECT
    l.value_date,
    tt.name                          AS transaction_type,
    CASE WHEN l.debit_amount > 0 THEN -l.debit_amount
         ELSE l.credit_amount END    AS amount,
    SUM(l.credit_amount - l.debit_amount)
        OVER (PARTITION BY l.account_id
              ORDER BY l.posted_at, l.entry_id
              ROWS UNBOUNDED PRECEDING) AS running_balance,
    l.description
```

```sql
FROM ledger l
JOIN transaction_types tt ON l.type_id = tt.type_id
WHERE l.account_id = 1
ORDER BY l.posted_at, l.entry_id;

-- ------------------------------------------------------------
-- Q3: Monthly interest accrual calculation
-- ------------------------------------------------------------
WITH balances AS (
    SELECT
        a.account_id,
        a.account_number,
        a.interest_rate,
        SUM(l.credit_amount - l.debit_amount) AS current_balance
    FROM accounts a
    JOIN account_types at2 ON a.type_id = at2.type_id
    LEFT JOIN ledger l     ON a.account_id = l.account_id
    WHERE at2.code IN ('SAVINGS','PREMIUM') AND a.status = 'Active'
    GROUP BY a.account_id, a.account_number, a.interest_rate
)
SELECT
    account_number,
    ROUND(current_balance, 2)                    AS balance,
    interest_rate * 100                          AS annual_rate_pct,
    ROUND(current_balance * interest_rate / 12, 4) AS monthly_interest,
    ROUND(current_balance * interest_rate / 365, 6) AS daily_interest
FROM balances
WHERE current_balance > 0
ORDER BY monthly_interest DESC;

-- ------------------------------------------------------------
-- Q4: Loan amortization schedule (12-month example)
-- ------------------------------------------------------------
WITH RECURSIVE amortization AS (
    SELECT
        1                           AS month_num,
        10000.0000                  AS principal_balance,
        0.0899 / 12                 AS monthly_rate,
        -- Monthly payment formula: P * r / (1 - (1+r)^-n)
        ROUND(10000.0000 * (0.0899/12) / (1 - POWER(1 + 0.0899/12, -36)), 4) AS
monthly_payment
    UNION ALL
    SELECT
        a.month_num + 1,
        ROUND(a.principal_balance - (a.monthly_payment - a.principal_balance *
a.monthly_rate), 4),
        a.monthly_rate,
        a.monthly_payment
    FROM amortization a
    WHERE a.month_num < 36 AND a.principal_balance > 0
)
SELECT
```

```
    month_num,
    ROUND(principal_balance, 2)                                AS balance_start,
    ROUND(principal_balance * monthly_rate, 2)                 AS interest_portion,
    ROUND(monthly_payment - principal_balance * monthly_rate, 2) AS
principal_portion,
    ROUND(monthly_payment, 2)                                  AS total_payment,
    ROUND(GREATEST(0, principal_balance
          - (monthly_payment - principal_balance * monthly_rate)), 2) AS balance_end
FROM amortization
ORDER BY month_num;

-- ------------------------------------------------------------
-- Q5: Fraud velocity check - transactions in last hour
-- ------------------------------------------------------------
SELECT
    a.account_number,
    c.first_name || ' ' || c.last_name   AS customer,
    COUNT(l.entry_id)                    AS txns_last_hour,
    SUM(l.debit_amount)                  AS total_debited_last_hour
FROM ledger l
JOIN accounts a  ON l.account_id  = a.account_id
JOIN customers c ON a.customer_id = c.customer_id
WHERE l.posted_at >= NOW() - INTERVAL '1 hour'
  AND l.debit_amount > 0
GROUP BY a.account_id, a.account_number, c.first_name, c.last_name
HAVING COUNT(l.entry_id) > 5 OR SUM(l.debit_amount) > 5000.0000
ORDER BY txns_last_hour DESC;

-- ------------------------------------------------------------
-- Q6: Customer wealth summary (all accounts)
-- ------------------------------------------------------------
SELECT
    c.customer_number,
    c.first_name || ' ' || c.last_name   AS customer,
    c.risk_tier,
    COUNT(DISTINCT a.account_id)          AS account_count,
    SUM(COALESCE(bal.balance, 0)) FILTER (WHERE at2.code IN
('SAVINGS','CHECKING','PREMIUM'))
                                          AS total_deposits,
    SUM(COALESCE(bal.balance, 0)) FILTER (WHERE at2.code = 'CREDIT')
                                          AS credit_balance,
    SUM(COALESCE(bal.balance, 0)) FILTER (WHERE at2.code = 'LOAN')
                                          AS loan_balance
FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id AND a.status = 'Active'
JOIN account_types at2 ON a.type_id = at2.type_id
LEFT JOIN LATERAL (
    SELECT SUM(credit_amount - debit_amount) AS balance
    FROM ledger WHERE account_id = a.account_id
) bal ON TRUE
GROUP BY c.customer_id, c.customer_number, c.first_name, c.last_name, c.risk_tier
ORDER BY total_deposits DESC NULLS LAST;
```

```sql
-- ------------------------------------------------------------
-- Q7: Branch performance summary
-- ------------------------------------------------------------
SELECT
    b.name                                  AS branch,
    COUNT(DISTINCT c.customer_id)           AS customers,
    COUNT(DISTINCT a.account_id)            AS accounts,
    ROUND(SUM(CASE WHEN l.credit_amount > 0 THEN l.credit_amount ELSE 0 END), 2) AS
total_deposits_posted,
    ROUND(SUM(CASE WHEN l.debit_amount > 0 THEN l.debit_amount ELSE 0 END), 2)   AS
total_withdrawals_posted
FROM branches b
LEFT JOIN customers c ON b.branch_id = c.branch_id
LEFT JOIN accounts a  ON c.customer_id = a.customer_id
LEFT JOIN ledger l    ON a.account_id = l.account_id
    AND l.value_date >= DATE_TRUNC('month', CURRENT_DATE)
GROUP BY b.branch_id, b.name
ORDER BY total_deposits_posted DESC NULLS LAST;

-- ------------------------------------------------------------
-- Q8: Transfer reconciliation
-- ------------------------------------------------------------
SELECT
    t.ref_number,
    a_from.account_number               AS from_account,
    a_to.account_number                 AS to_account,
    t.amount,
    t.fee_amount,
    t.status,
    t.initiated_at::DATE,
    t.completed_at::DATE,
    CASE WHEN t.status = 'Completed'
         THEN (t.completed_at - t.initiated_at)
         ELSE NULL
    END                                 AS processing_time
FROM transfers t
JOIN accounts a_from ON t.from_account_id = a_from.account_id
JOIN accounts a_to   ON t.to_account_id   = a_to.account_id
ORDER BY t.initiated_at DESC;

-- ------------------------------------------------------------
-- Q9: Unresolved fraud signals by severity
-- ------------------------------------------------------------
SELECT
    fs.severity,
    COUNT(*)                            AS signal_count,
    COUNT(DISTINCT fs.account_id)       AS affected_accounts,
    MIN(fs.created_at)::DATE            AS oldest_signal,
    MAX(fs.created_at)::DATE            AS newest_signal
FROM fraud_signals fs
WHERE fs.is_resolved = FALSE
```

```
GROUP BY fs.severity
ORDER BY CASE fs.severity WHEN 'Critical' THEN 1 WHEN 'High' THEN 2
                          WHEN 'Medium' THEN 3 ELSE 4 END;

-- ------------------------------------------------------
-- Q10: Balance sheet totals (double-entry verification)
-- ------------------------------------------------------
SELECT
    ca.type                             AS account_type,
    ca.name                             AS coa_account,
    ROUND(SUM(l.debit_amount), 2)       AS total_debits,
    ROUND(SUM(l.credit_amount), 2)      AS total_credits,
    ROUND(SUM(l.credit_amount - l.debit_amount), 2) AS net_balance
FROM ledger l
JOIN coa_accounts ca ON l.coa_id = ca.coa_id
WHERE l.value_date >= DATE_TRUNC('month', CURRENT_DATE)
GROUP BY ca.coa_id, ca.type, ca.name
ORDER BY ca.type, ca.name;
```

---

9. Stored Procedures / Functions

```
-- ------------------------------------------------------
-- FUNCTION 1: Execute a funds transfer (SERIALIZABLE)
-- ------------------------------------------------------
CREATE OR REPLACE FUNCTION bank.transfer_funds(
    p_from_account_id  INTEGER,
    p_to_account_id    INTEGER,
    p_amount           NUMERIC(18,4),
    p_description      VARCHAR DEFAULT 'Transfer',
    p_initiated_by     VARCHAR DEFAULT current_user
)
RETURNS BIGINT
LANGUAGE plpgsql AS $$
DECLARE
    v_from_balance   NUMERIC(18,4);
    v_overdraft      NUMERIC(18,4);
    v_transfer_id    BIGINT;
    v_ref            VARCHAR(40);
    v_type_out_id    INTEGER;
    v_type_in_id     INTEGER;
BEGIN
    IF p_amount <= 0 THEN RAISE EXCEPTION 'Transfer amount must be positive'; END
IF;
    IF p_from_account_id = p_to_account_id THEN
        RAISE EXCEPTION 'Cannot transfer to same account';
    END IF;

    -- Acquire advisory locks ordered by account_id to prevent deadlock
    PERFORM pg_advisory_xact_lock(LEAST(p_from_account_id, p_to_account_id));
    PERFORM pg_advisory_xact_lock(GREATEST(p_from_account_id, p_to_account_id));
```

```
    -- Get current balance of from account
    SELECT SUM(credit_amount - debit_amount) INTO v_from_balance
    FROM bank.ledger WHERE account_id = p_from_account_id;

    -- Get overdraft limit
    SELECT at2.overdraft_limit INTO v_overdraft
    FROM bank.accounts a
    JOIN bank.account_types at2 ON a.type_id = at2.type_id
    WHERE a.account_id = p_from_account_id;

    IF COALESCE(v_from_balance, 0) + COALESCE(v_overdraft, 0) < p_amount THEN
        RAISE EXCEPTION 'Insufficient funds. Balance: %, Overdraft: %, Requested:
%',
            v_from_balance, v_overdraft, p_amount;
    END IF;

    -- Get transaction type IDs
    SELECT type_id INTO v_type_out_id FROM bank.transaction_types WHERE code =
'TRANSFER_OUT';
    SELECT type_id INTO v_type_in_id  FROM bank.transaction_types WHERE code =
'TRANSFER_IN';

    v_ref := 'TRF-' || gen_random_uuid()::TEXT;

    -- Insert transfer record
    INSERT INTO bank.transfers (ref_number, from_account_id, to_account_id, amount,
description,
                                initiated_by, status)
    VALUES (v_ref, p_from_account_id, p_to_account_id, p_amount, p_description,
            p_initiated_by, 'Pending')
    RETURNING transfer_id INTO v_transfer_id;

    -- Debit from account
    INSERT INTO bank.ledger (transaction_ref, account_id, type_id, debit_amount,
                             description, reference_id, reference_type)
    VALUES (v_ref, p_from_account_id, v_type_out_id, p_amount,
            p_description, v_transfer_id, 'TRANSFER');

    -- Credit to account
    INSERT INTO bank.ledger (transaction_ref, account_id, type_id, credit_amount,
                             description, reference_id, reference_type)
    VALUES (v_ref, p_to_account_id, v_type_in_id, p_amount,
            p_description, v_transfer_id, 'TRANSFER');

    -- Mark transfer complete
    UPDATE bank.transfers
    SET status = 'Completed', completed_at = NOW()
    WHERE transfer_id = v_transfer_id;

    -- Update last activity
    UPDATE bank.accounts
```

```
    SET last_activity = NOW()
    WHERE account_id IN (p_from_account_id, p_to_account_id);

    RETURN v_transfer_id;
END;
$$;

-- ------------------------------------------------------
-- FUNCTION 2: Run monthly interest posting
-- ------------------------------------------------------
CREATE OR REPLACE FUNCTION bank.post_monthly_interest(p_month DATE DEFAULT
DATE_TRUNC('month', CURRENT_DATE)::DATE)
RETURNS TABLE (account_number VARCHAR, interest_posted NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    v_acct   RECORD;
    v_balance NUMERIC(18,4);
    v_interest NUMERIC(18,4);
    v_type_id INTEGER;
    v_ref     VARCHAR;
BEGIN
    SELECT type_id INTO v_type_id FROM bank.transaction_types WHERE code =
'INTEREST';

    FOR v_acct IN
        SELECT a.account_id, a.account_number, a.interest_rate
        FROM bank.accounts a
        JOIN bank.account_types at2 ON a.type_id = at2.type_id
        WHERE at2.code IN ('SAVINGS','CHECKING','PREMIUM')
          AND a.status = 'Active'
          AND a.interest_rate > 0
          AND NOT EXISTS (
              SELECT 1 FROM bank.ledger l2
              JOIN bank.transaction_types tt ON l2.type_id = tt.type_id
              WHERE l2.account_id = a.account_id
                AND tt.code = 'INTEREST'
                AND DATE_TRUNC('month', l2.value_date) = p_month
          )
    LOOP
        SELECT SUM(credit_amount - debit_amount) INTO v_balance
        FROM bank.ledger WHERE account_id = v_acct.account_id
          AND value_date < p_month + INTERVAL '1 month';

        IF COALESCE(v_balance, 0) <= 0 THEN CONTINUE; END IF;

        v_interest := ROUND(v_balance * v_acct.interest_rate / 12, 4);
        v_ref := 'INT-' || TO_CHAR(p_month, 'YYYYMM') || '-' || v_acct.account_id;

        INSERT INTO bank.ledger (transaction_ref, account_id, type_id,
credit_amount,
                                 description, value_date)
        VALUES (v_ref, v_acct.account_id, v_type_id, v_interest,
```

```
                'Monthly interest for ' || TO_CHAR(p_month, 'Month YYYY'),
                (p_month + INTERVAL '1 month - 1 day')::DATE);

        RETURN QUERY SELECT v_acct.account_number::VARCHAR, v_interest;
    END LOOP;
END;
$$;

-- ------------------------------------------------------
-- FUNCTION 3: Fraud velocity check
-- ------------------------------------------------------
CREATE OR REPLACE FUNCTION bank.check_fraud_velocity(p_account_id INTEGER)
RETURNS TABLE (risk_level TEXT, reason TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_txns_1h      INTEGER;
    v_amount_1h    NUMERIC;
    v_txns_24h     INTEGER;
    v_large_txn    NUMERIC;
BEGIN
    SELECT COUNT(*), SUM(debit_amount) INTO v_txns_1h, v_amount_1h
    FROM bank.ledger
    WHERE account_id = p_account_id
      AND posted_at >= NOW() - INTERVAL '1 hour'
      AND debit_amount > 0;

    SELECT COUNT(*) INTO v_txns_24h
    FROM bank.ledger
    WHERE account_id = p_account_id
      AND posted_at >= NOW() - INTERVAL '24 hours'
      AND debit_amount > 0;

    SELECT MAX(debit_amount) INTO v_large_txn
    FROM bank.ledger
    WHERE account_id = p_account_id
      AND posted_at >= NOW() - INTERVAL '1 hour';

    IF v_txns_1h > 10 THEN
        RETURN QUERY SELECT 'Critical'::TEXT, format('%s transactions in 1 hour',
v_txns_1h);
    END IF;
    IF v_amount_1h > 10000 THEN
        RETURN QUERY SELECT 'High'::TEXT, format('$%s debited in last hour',
v_amount_1h);
    END IF;
    IF v_txns_24h > 30 THEN
        RETURN QUERY SELECT 'Medium'::TEXT, format('%s transactions in 24 hours',
v_txns_24h);
    END IF;
    IF v_large_txn > 5000 THEN
        RETURN QUERY SELECT 'Medium'::TEXT, format('Large single transaction: $%s',
v_large_txn);
```

```
    END IF;
END;
$$;
```

---

10. Triggers

```
-- TRIGGER 1: Prevent ledger updates or deletes (immutability)
CREATE OR REPLACE FUNCTION bank.trg_ledger_immutable()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION 'Ledger entries are immutable. Use REVERSAL entries
instead.';
    END IF;
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION 'Ledger entries cannot be deleted. Regulatory retention
required.';
    END IF;
    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_ledger_immutability
    BEFORE UPDATE OR DELETE ON bank.ledger
    FOR EACH ROW EXECUTE FUNCTION bank.trg_ledger_immutable();

-- TRIGGER 2: Auto-create fraud signal on large transaction
CREATE OR REPLACE FUNCTION bank.trg_large_txn_fraud_check()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.debit_amount >= 10000.0000 THEN
        INSERT INTO bank.fraud_signals (account_id, signal_type, severity,
description, raw_data)
        VALUES (NEW.account_id, 'LARGE_TRANSACTION', 'High',
                format('Transaction of $%s exceeds $10,000 threshold',
NEW.debit_amount),
                jsonb_build_object('entry_id', NEW.entry_id, 'amount',
NEW.debit_amount));
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_fraud_large_txn
    AFTER INSERT ON bank.ledger
    FOR EACH ROW EXECUTE FUNCTION bank.trg_large_txn_fraud_check();
```

---

11. Performance Optimization

```sql
-- Row-Level Security for customer data isolation
ALTER TABLE bank.accounts ENABLE ROW LEVEL SECURITY;

CREATE POLICY accounts_customer_policy ON bank.accounts
    FOR SELECT
    USING (customer_id = current_setting('app.current_customer_id', TRUE)::INTEGER
           OR current_setting('app.role', TRUE) IN ('banker','admin'));

-- Covering index for statement queries
CREATE INDEX idx_ledger_account_covering
    ON bank.ledger (account_id, posted_at DESC)
    INCLUDE (debit_amount, credit_amount, description, value_date);

-- Partial index for pending transfers
CREATE INDEX idx_transfers_pending
    ON bank.transfers (initiated_at)
    WHERE status = 'Pending';
```

Technique	Why
Ledger partitioned by year	Balance queries on recent data only scan one partition
Covering index on ledger	Statement query never hits heap for common columns
Advisory locks by account ID order	Prevents deadlocks in concurrent transfers
SERIALIZABLE isolation	Prevents phantom reads during balance checks
RLS on accounts	Customers cannot see other customers' data

---

12. Extensions Used

```sql
CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- gen_random_uuid(), hash functions
CREATE EXTENSION IF NOT EXISTS pg_cron;     -- schedule interest posting
CREATE EXTENSION IF NOT EXISTS pg_stat_statements; -- monitor query performance

-- Schedule monthly interest on the 1st of every month
SELECT cron.schedule('monthly-interest', '0 2 1 * *',
    'SELECT * FROM bank.post_monthly_interest()');
```

---

13. Testing Guide

```sql
-- TEST 1: Transfer funds between accounts
SELECT bank.transfer_funds(1, 2, 500.0000, 'Transfer from savings to checking');

-- Verify balances
SELECT account_id, SUM(credit_amount - debit_amount) AS balance
FROM bank.ledger GROUP BY account_id;
```

```sql
-- TEST 2: Attempt overdraft (should fail with insufficient funds)
SELECT bank.transfer_funds(2, 1, 9999999.0000, 'Should fail');

-- TEST 3: Self-transfer prevention
SELECT bank.transfer_funds(1, 1, 100.0000);  -- Should fail

-- TEST 4: Ledger immutability
UPDATE bank.ledger SET credit_amount = 999999 WHERE entry_id = 1;  -- Should raise
error

-- TEST 5: Monthly interest posting
SELECT * FROM bank.post_monthly_interest();

-- TEST 6: Fraud velocity check
SELECT * FROM bank.check_fraud_velocity(1);

-- TEST 7: Large transaction fraud trigger
INSERT INTO bank.ledger (transaction_ref, account_id, type_id, debit_amount,
description)
VALUES ('TEST-LARGE', 1, 2, 12000.0000, 'Large test withdrawal');
SELECT * FROM bank.fraud_signals WHERE account_id = 1 ORDER BY created_at DESC LIMIT
1;
-- Should show High severity signal
```

---

14. Extension Challenges

1. **SWIFT / SEPA Wire Processing**: Add an `external_banks` table with SWIFT/BIC codes. Implement international wire transfers with FX conversion rates. Write an end-of-day settlement process that nets interbank positions.
2. **Credit Scoring**: Build a `credit_scores` table updated monthly based on account behavior (payment history, utilization, account age). Write a function that computes a 300-850 score from ledger data.
3. **Regulatory Reporting (BSA/AML)**: Add a `sar_reports` table (Suspicious Activity Reports). Write queries that identify structuring (multiple transactions just below $10,000 threshold). Generate a monthly CTR (Currency Transaction Report) for large cash transactions.
4. **Multi-Currency Support**: Extend transfers to support cross-currency with exchange rates stored in `fx_rates`. Implement a FX conversion that records both the source and destination amounts with the rate used.
5. **Account Statements PDF Queue**: Add a `statement_queue` table. Write a function that generates monthly statement data per account (all ledger entries, opening/closing balance, interest earned). Simulate a batch job that marks statements as generated.

---

15. What You Learned

Skill	Demonstrated By

Double-entry bookkeeping	Every transfer creates two immutable ledger entries
NUMERIC precision	`NUMERIC(18,4)` for all money — zero float errors
Ledger immutability	Trigger prevents UPDATE/DELETE on ledger
Advisory locking	`pg_advisory_xact_lock` ordered by ID to prevent deadlock
Loan amortization	Recursive CTE calculates monthly payment schedule
Row-Level Security	Customers can only see their own account data
Fraud detection	Velocity checks via aggregate queries
SERIALIZABLE isolation	Prevents phantom reads in balance checks
Partitioning	Ledger partitioned by year for query performance
Interest computation	Monthly posting derived from annual rate / 12